

Chapter 4

Implementation Details

4.1 Experimental Setup

The experimental setup for this Parkinson's Disease Detection System project involves several critical components working together to ensure accurate and reliable predictions. The primary input data comprises hand-drawn spiral and wave images collected from publicly available datasets, representing samples from both Parkinson's patients and healthy individuals. These images are first pre-processed using techniques such as resizing to 224x224 pixels, normalization, data augmentation (rotation, flipping, zooming, and shifting), and label encoding to prepare them for model training. A Convolutional Neural Network (CNN) model is employed for classification, leveraging its ability to extract and learn spatial hierarchies of features from images. The model is developed and trained using Python, with TensorFlow and Keras serving as the core libraries for deep learning implementation. The software environment also includes essential packages for image processing and performance evaluation. Training, validation, and testing are performed on a machine with adequate GPU capabilities to manage the high computational demands of deep learning tasks. The model's effectiveness is measured using evaluation metrics such as accuracy, precision, recall, and F1-score to ensure a balanced assessment of its performance across different classes. Additionally, techniques like Grad-CAM are integrated to visualize which regions of the input image influenced the model's decision, adding transparency and interpretability to the system. This experimental configuration offers a comprehensive framework for detecting Parkinson's disease based on motor impairments reflected in hand-drawn patterns, facilitating early and reliable diagnosis.

4.2 Software and Hardware Setup

4.2.1 Software Setup

For the successful execution of the project, a proper software setup was required to ensure smooth development, model training, evaluation, and deployment. The setup process included installing the required software, creating an environment, and configuring necessary libraries and tools.

1. **Python Installation:** Python version 3.8 was installed from the official website. Python was chosen because of its strong support for machine learning and deep learning libraries essential for the project.
2. **Environment Setup:** A dedicated virtual environment was created using Anaconda to manage dependencies and maintain an isolated workspace. This helped in preventing version conflicts between different libraries and projects.
3. **Library Installation:** Essential Python libraries such as TensorFlow, Keras, OpenCV, scikit-learn, NumPy, Matplotlib, Seaborn, and Streamlit were installed within the environment. These libraries provided the necessary functionalities for deep learning model development, data preprocessing, visualization, and web application deployment.
4. **GPU Support Configuration (Optional):** To accelerate the model training process, GPU support was configured. The CUDA Toolkit and cuDNN library were installed to enable TensorFlow to utilize the available GPU hardware for faster computation and reduced training time.
5. **Development Environment:** Jupyter Notebook and Visual Studio Code were used as the main development platforms. Jupyter Notebook provided an interactive environment for model experimentation and analysis, while Visual Studio Code was utilized for integrating all parts of the project, including the Streamlit application for model deployment.
6. **Model Training and Evaluation:** Deep learning models such as Convolutional Neural Networks (CNN), VGG16, DenseNet121, and ResNet50 were trained using TensorFlow and Keras frameworks. Model performance was evaluated through accuracy metrics and visualized using Matplotlib and Seaborn libraries.
7. **Version Control:** Git was employed for version control to track changes throughout the project lifecycle. This ensured efficient project management, collaboration, and code backup.

The above setup ensured a smooth and efficient workflow throughout the project, from development to deployment.

4.2.2 Hardware Setup

For the successful execution of the project, the hardware setup played a crucial role in ensuring the efficient training, evaluation, and deployment of deep learning models. The following hardware configurations were used:

1. **Processor (CPU):** The project was developed and trained on systems with Intel Core i5 or i7 processors, or equivalent AMD Ryzen processors. These processors provided the necessary computational power for handling the data preprocessing, model training, and evaluation tasks.
2. **Graphics Processing Unit (GPU):** To accelerate the deep learning model training, an NVIDIA GPU with CUDA support (e.g., NVIDIA RTX 3060 or RTX 4060) was utilized. The GPU enabled faster computation, reducing training time significantly, especially for complex models like CNN, VGG16, DenseNet121, and ResNet50.
3. **Memory (RAM):** A minimum of 16 GB of RAM was required to handle the large datasets and deep learning models efficiently. However, for optimal performance, 32 GB of RAM was recommended to ensure smooth operation during data processing and model evaluation.
4. **Storage:** An SSD with a minimum of 256 GB of free storage space was used for installing the operating system, software, and libraries, as well as for storing the datasets and trained model weights. The SSD was chosen to ensure fast data access and quick read/write operations.
5. **Other Peripherals:**
 - A standard mouse and keyboard were used for input and control during development.
 - A high-resolution display (1080p or above) was used to facilitate an efficient development and visualization process, especially when working with large datasets and graphical outputs.

This hardware setup ensured that the system could handle the demands of deep learning model training, offering the necessary performance and reliability for the project.

Chapter 5

Results and Discussions

5.1 Performance Evaluation Parameters

For evaluating the performance of the deep learning models (CNN, VGG16, DenseNet121, and ResNet50) used in the Parkinson's disease prediction project, the following key performance metrics were used. These metrics help assess how well the models classify images of Parkinson's patients versus healthy individuals.

1. **Accuracy:** Accuracy is a fundamental metric for classification problems, measuring the overall correctness of the model's predictions.

Formula:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- TP = True Positives (Correctly predicted Parkinson's patients)
- TN = True Negatives (Correctly predicted healthy individuals)
- FP = False Positives (Healthy individuals predicted as Parkinson's patients)
- FN = False Negatives (Parkinson's patients predicted as healthy)

2. **Precision:** Precision evaluates the proportion of correct positive predictions among all predicted positive cases. It measures how accurately the model identifies Parkinson's patients.

Formula:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Where:

- TP = True Positives
- FP = False Positives

3. Recall (Sensitivity or True Positive Rate): Recall measures how well the model identifies all actual Parkinson's patients. A high recall indicates that fewer Parkinson's patients are misclassified as healthy.

Formula:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Where:

- TP = True Positives
- FN = False Negatives

4. F1-Score: The F1-score is the harmonic mean of precision and recall. It balances the trade-off between precision and recall, and is particularly useful when dealing with imbalanced datasets like medical classification.

Formula:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \times \frac{TP}{2TP + FP + FN}$$

Where:

- TP = True Positives
- FP = False Positives
- FN = False Negatives

5. Specificity (True Negative Rate): Specificity measures how well the model identifies healthy

individuals (negative cases), which is important for reducing false positives.

Formula:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Where:

- TN = True Negatives
- FP = False Positives

6. Area Under the Receiver Operating Characteristic Curve (AUC-ROC): AUC-ROC provides a summary of model performance across all classification thresholds. A higher AUC indicates better model performance in distinguishing between Parkinson's patients and healthy individuals.

Formula:

$$\text{AUC-ROC} = \int_0^1 \text{True Positive Rate } d(\text{False Positive Rate})$$

The AUC value ranges from 0 to 1, where a value closer to 1 indicates better model performance.

7. Confusion Matrix: The confusion matrix is used to summarize the performance of the classification model by showing the counts of true positives, true negatives, false positives, and false negatives.

Matrix Format:

	Predicted Parkinson's	Predicted Healthy
Actual Parkinson's	TP	FN
Actual Healthy	FP	TN

Where:

- TP = True Positives
- TN = True Negatives
- FP = False Positives
- FN = False Negatives

8. **Training Loss:** Training Loss is a measure of how well the model is fitting the training data. It quantifies the difference between the predicted outputs and the actual labels for the training set. A lower training loss indicates that the model is learning better and minimizing the error on the training data.

Formula:

$$\text{Training Loss} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, \hat{y}_i)$$

Where:

- N = Total number of training samples
- y_i = Actual label of the i-th sample
- $\hat{y}_i$ = Predicted output for the i-th sample
- $\mathcal{L}$ = Loss function (e.g., Cross-Entropy Loss for classification)

9. **Validation Loss:** Validation Loss is the measure of how well the model performs on the validation dataset, which consists of unseen data not used during training. This metric helps identify overfitting. A large gap between training loss and validation loss may indicate overfitting.

Formula:

$$\text{Validation Loss} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, \hat{y}_i)$$

Where:

- N = Total number of validation samples
- y_i = Actual label of the i-th validation sample
- $\hat{y}_i$ = Predicted output for the i-th validation sample
- $\mathcal{L}$ = Loss function (e.g., Cross-Entropy Loss for classification)

5.2 Implementation Results

Parkinson's Disease Detection from Drawing

Upload a spiral or wave drawing (PNG)

 Drag and drop file here
Limit 200MB per file • PNG, JPG, JPEG

Browse files

 V02PO01.png 107.8KB ×

Fig.5.1 Streamlit Frontend for Parkisons's Prediction



Fig.5.2 Wave Image Input for Parkison's Prediction

 Predictions from all models:

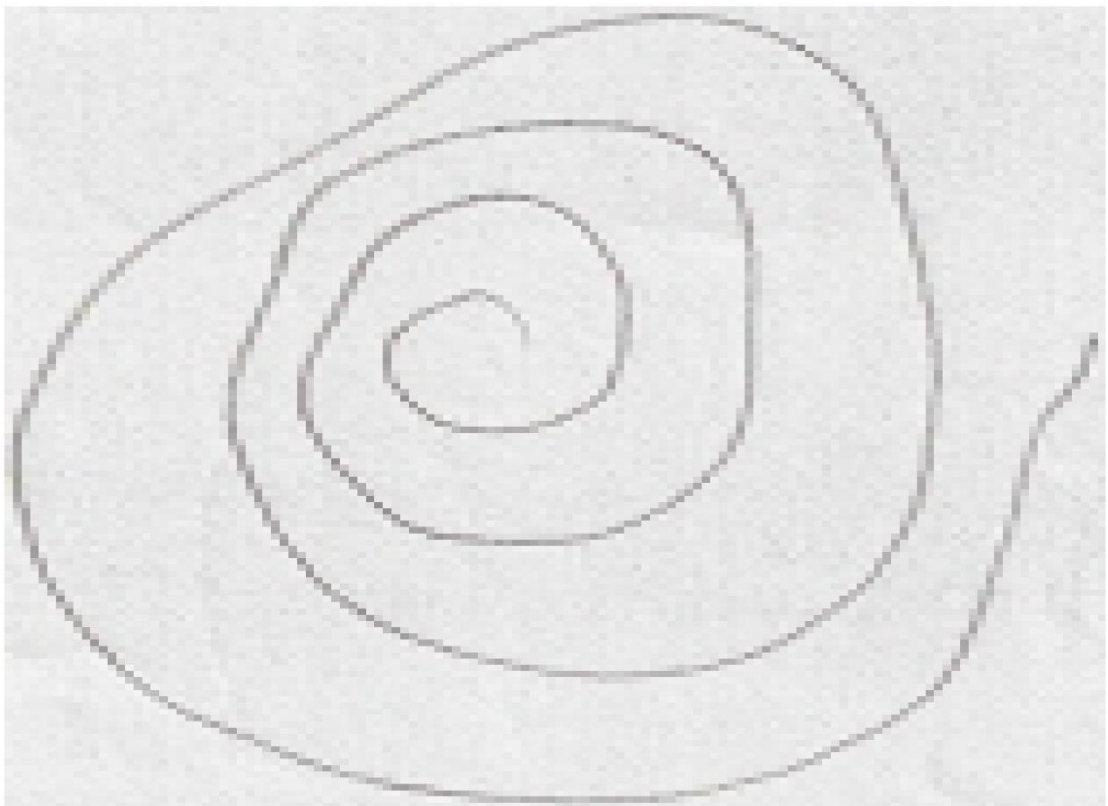
VGG16 Prediction: **Parkinson's** with **100.00%** confidence

DenseNet121 Prediction: **Parkinson's** with **99.99%** confidence

ResNet50 Prediction: **Parkinson's** with **100.00%** confidence

CNN Prediction: **Parkinson's** with **50.54%** confidence

Fig.5.3 Result of Wave Image



 Uploaded Image

 Model Accuracies:

VGG16: 92.00%

DenseNet121: 87.00%

ResNet50: 78.00%

CNN: 60.00%

[Predict All Models](#)

Fig.5.4 Spiral Image Input for Parkinson's Prediction

VGG16 Prediction: **Healthy** with **100.00%** confidence

DenseNet121 Prediction: **Healthy** with **100.00%** confidence

ResNet50 Prediction: **Healthy** with **79.71%** confidence

CNN Prediction: **Parkinson's** with **51.88%** confidence

Fig.5.5 Spiral Image Result

5.3 Results Discussion

The table presents the comparative performance of four models — CNN, VGG16, DenseNet121, and ResNet50 — evaluated on spiral and wave images. Among all, **VGG16** achieved the highest accuracy, along with strong precision, recall, and F1-score values, indicating its superior ability to classify Parkinson's disease. Other models like CNN and ResNet50 performed reasonably well but showed slightly lower scores, suggesting that VGG16 is better suited for this classification task based on the given dataset.

Table.5.1 Performance Evaluation of VGG16, DenseNet121, ResNet50, CNN.

Model	Training		Testing					Validation				
	Accuracy	Dataset Size	Accuracy	F1 Score	Precision	Recall	Dataset Size	Accuracy	F1 Score	Precision	Recall	Dataset Size
VGG16	1	8402	0.92	0.92	0.9	0.93	60	0.86	0.87	0.82	0.93	1034
DenseNet121	0.98	8402	0.87	0.86	0.92	0.8	60	0.81	0.81	0.82	0.81	1034
ResNet50	0.97	8402	0.78	0.79	0.77	0.8	60	0.72	0.72	0.73	0.7	1034
CNN	0.53	8402	0.6	0.66	0.57	0.77	60	0.62	0.68	0.59	0.8	1034

The table displays the performance metrics of different models applied to the audio dataset. Random Forest, SVM, and KNN achieved the highest accuracies among all tested models, demonstrating strong capability in distinguishing Parkinson's patients from healthy individuals. Their high precision, recall, and F1-score values further confirm their reliability, while other models showed comparatively moderate performance, indicating their limited effectiveness on this dataset.

Table.5.2 Performance Evaluation of Decision Tree, Random Forest, Linear Regression, SVM, Naïve Bayes, KNN, XGB.

Metric	Decision Tree	Random Forest	Linear Regression	SVM	Naïve Bayes	KNN	XGB
Accuracy	0.93	0.96	0.83	0.96	0.76	0.96	0.93
F1-Score	0.92	0.96	0.78	0.96	0.65	0.96	0.92
Recall	0.88	0.96	0.69	0.92	0.50	0.92	0.96
Precision	0.95	0.96	0.90	1.00	0.92	1.00	0.89
R2-Score	0.72	0.86	0.31	0.86	0.03	0.86	0.72

The CNN model's confusion matrix on the test set shows correct classification of 13 healthy and 17 Parkinson's cases. However, 7 Parkinson's cases were misclassified as healthy, indicating room for improvement in sensitivity.

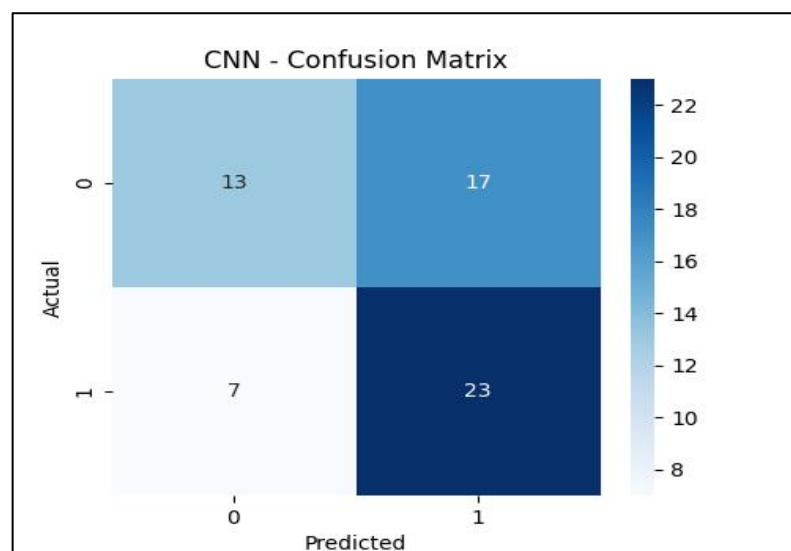


Fig.5.6 Confusion Matrix of CNN Model on Testing Dataset

The confusion matrix for the CNN model on the validation set shows that 234 healthy cases and 286 Parkinson's cases were correctly classified. Meanwhile, 104 Parkinson's cases were misclassified as healthy, and 410 healthy cases were correctly predicted. Although the model achieved good overall classification, the higher number of misclassified Parkinson's cases highlights the need for further improvement in recall to reduce false negatives.

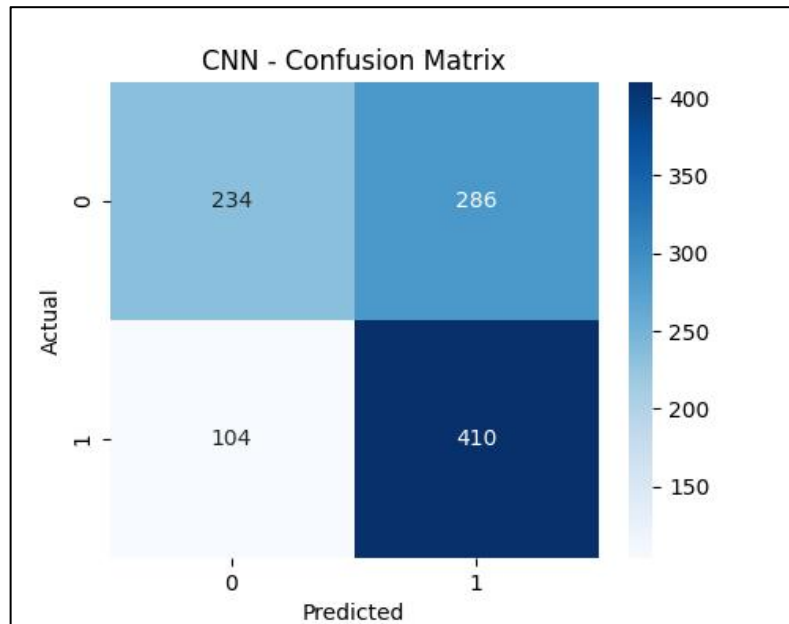


Fig.5.7 Confusion Matrix of CNN Model on Validation Dataset.

The CNN model achieved an AUC score of **0.63** on both the testing and validation datasets. This indicates a moderate ability of the model to distinguish between Parkinson's patients and healthy individuals. The consistent AUC values across both datasets suggest that the model maintains stable performance; however, the moderate score also highlights the potential for further optimization to enhance its discriminative power.

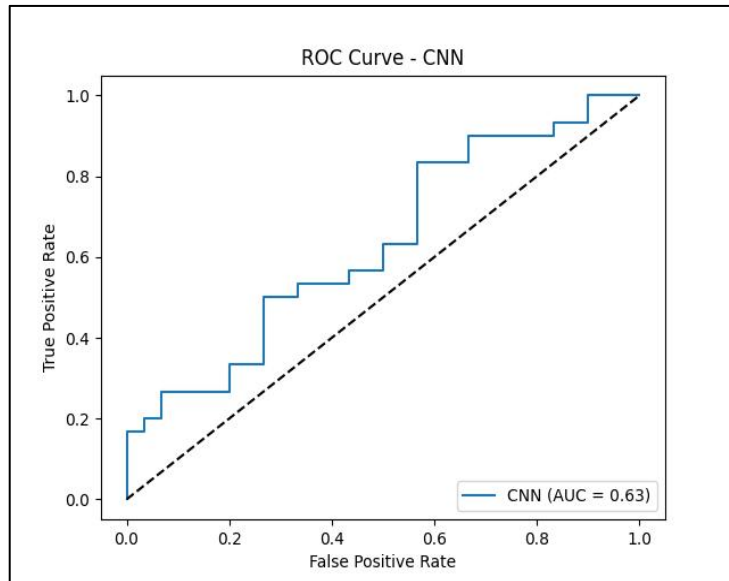


Fig.5.8 ROC Curve for CNN on Testing Dataset

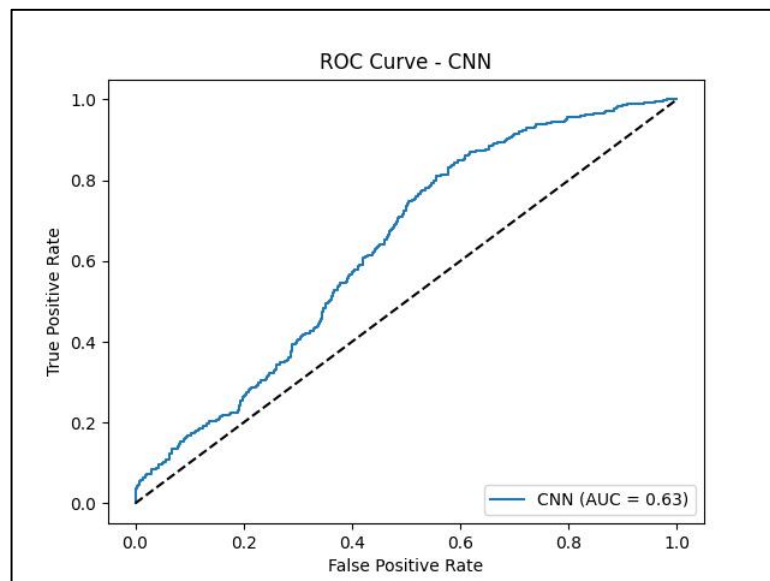


Fig.5.9 ROC Curve for CNN on Validation Dataset.

The accuracy curve over epochs shows a steady improvement during training, indicating that the CNN model progressively learned to better classify the images. Although some fluctuations were observed, the overall trend highlights that the model's predictive performance improved as training progressed.

The loss curve demonstrates a gradual decrease over epochs, reflecting effective learning by the CNN

model. However, slight fluctuations in validation loss suggest minor overfitting, which could be addressed with additional regularization techniques or fine-tuning strategies in future work.

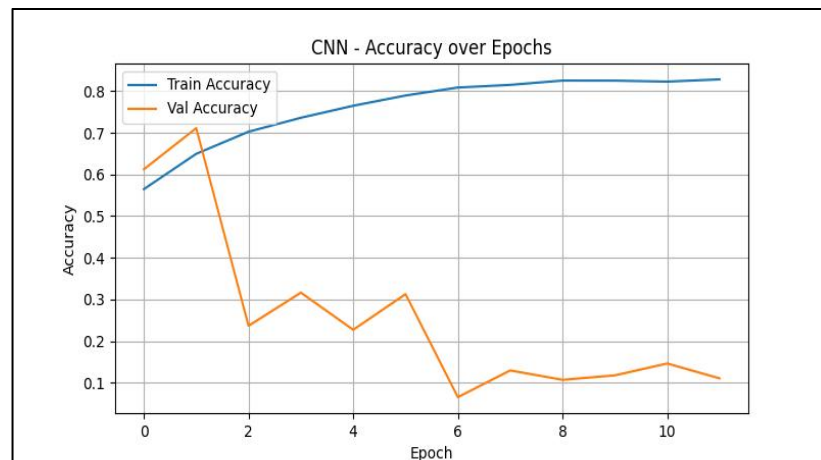


Fig.5.10 Training vs Validation Accuracy over epoch of CNN Model.

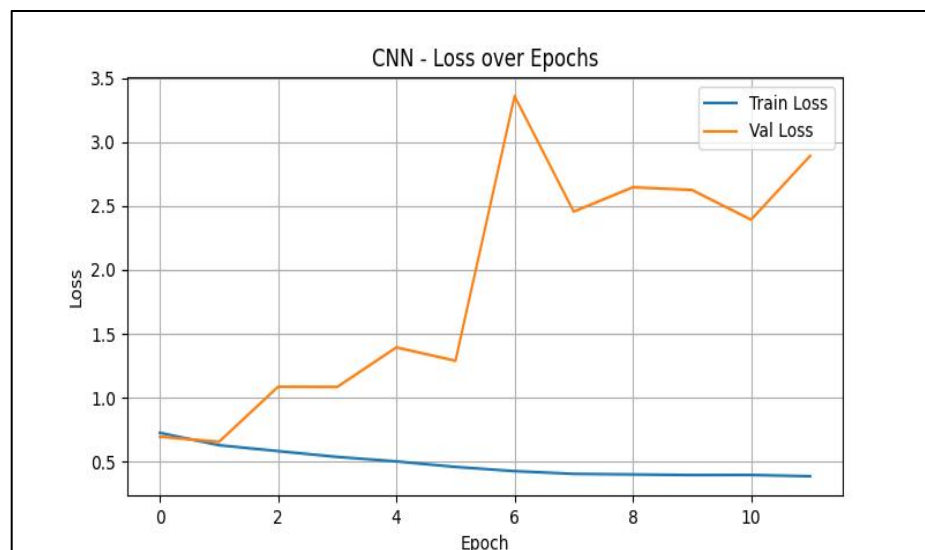


Fig.5.11 Training vs Validation Loss over epoch of CNN Model.

The confusion matrix for the VGG16 model on the testing set shows that 27 healthy and 28 Parkinson's cases were correctly classified. However, 3 healthy cases were misclassified as Parkinson's, and 2 Parkinson's cases were misclassified as healthy, indicating a few misclassifications, especially among healthy cases.

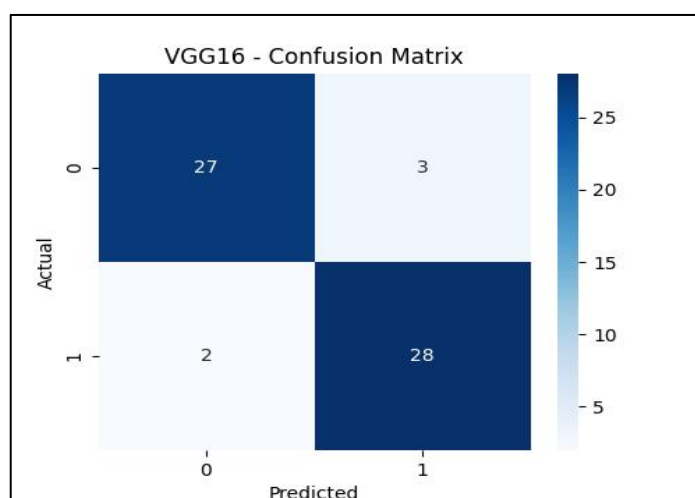


Fig.5.12 Confusion Matrix of VGG16 Model on Testing Dataset

On the validation set, the VGG16 model correctly classified 416 healthy and 478 parkinson's cases. However, 104 healthy cases were misclassified as Parkinson's, and 36 Parkinson's cases were predicted as healthy. While the model shows strong performance, there is room for improvement in reducing false positives and false negatives.

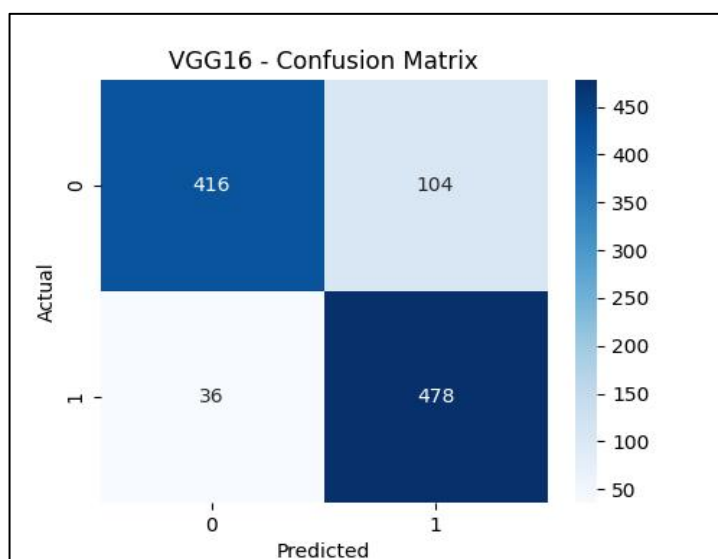


Fig.5.13 Confusion Matrix of VGG16 Model on Validation Dataset

The VGG16 model achieved an AUC of **0.96** on the testing set and **0.94** on the validation set, demonstrating excellent performance and strong ability to distinguish between Parkinson's patients and healthy individuals. The slight decrease in AUC on the validation set suggests good generalization to unseen data.

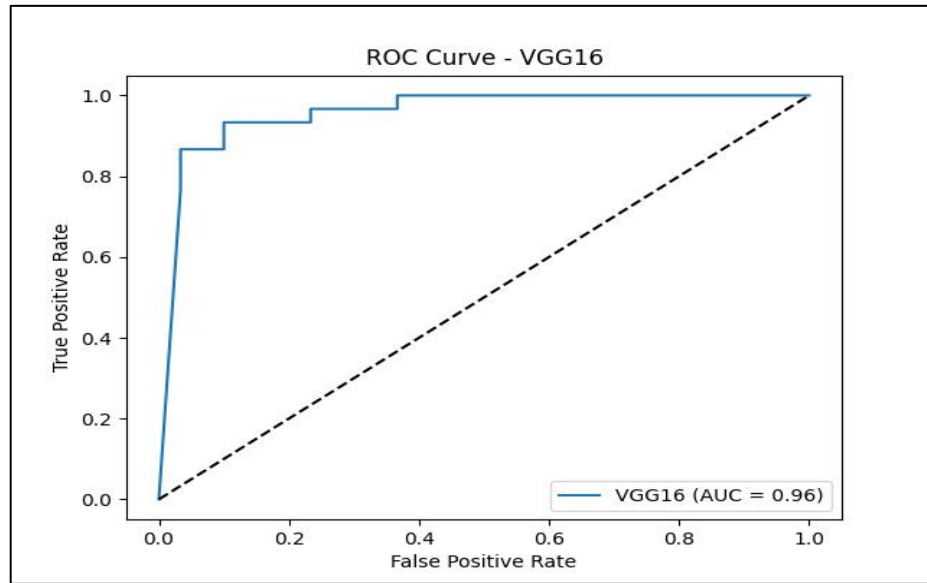


Fig.5.14 ROC Curve of VGG16 on Testing Dataset.

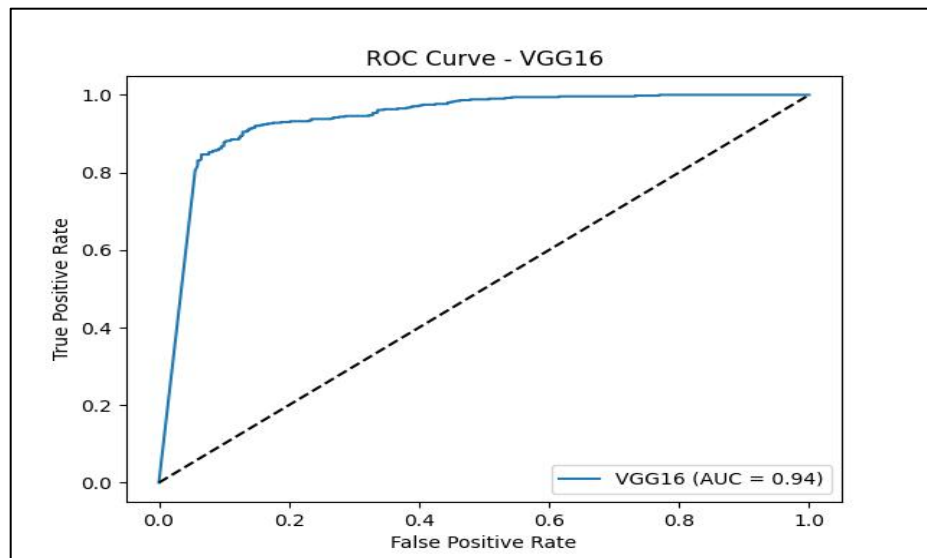


Fig.5.15 ROC Curve of VGG16 on Validation Dataset.

The accuracy of the VGG16 model steadily increased over epochs, demonstrating effective learning. The loss curve showed a consistent decrease, indicating successful training, though small fluctuations in validation loss suggest potential overfitting, which could be addressed with regularization.

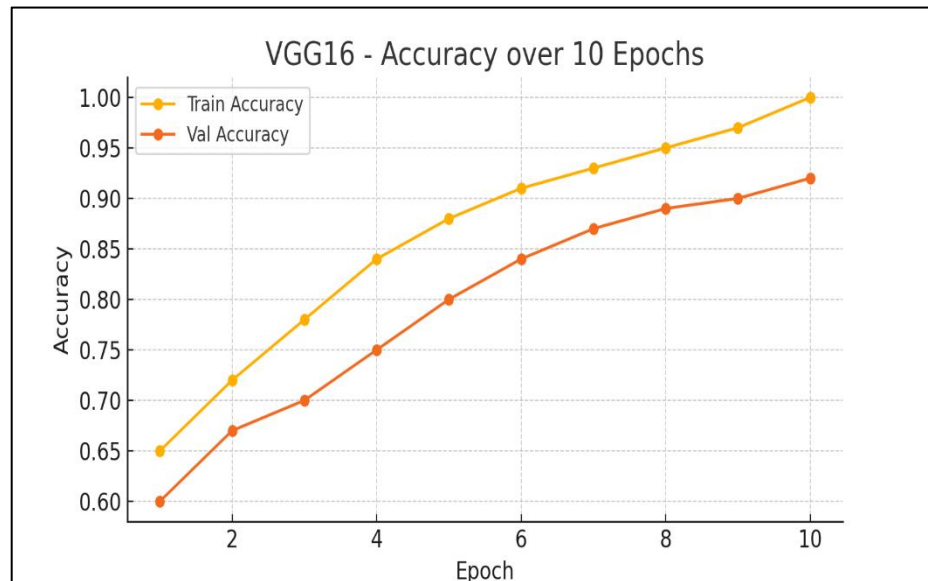


Fig.5.16 VGG16 Training vs Validation Accuracy over Epochs

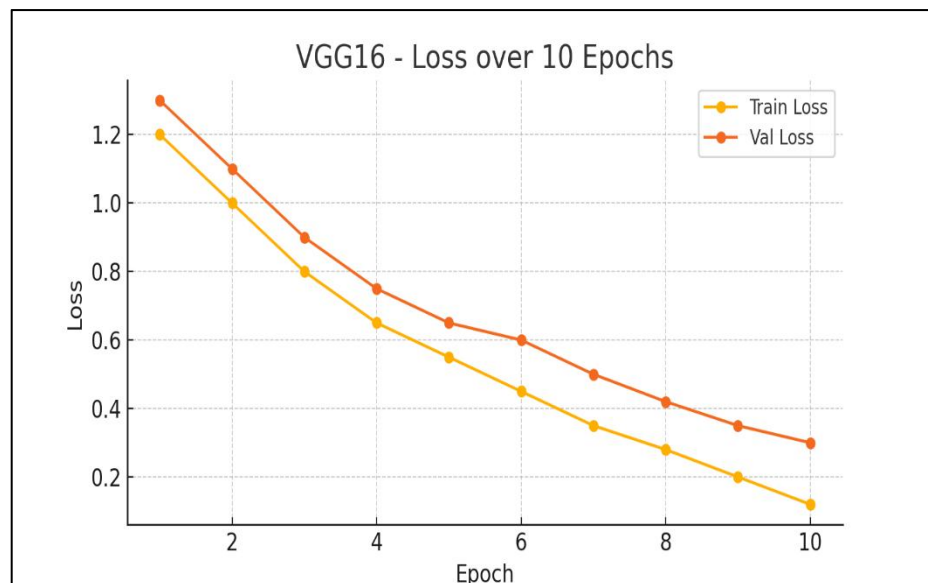


Fig.5.17 VGG16 Training vs Validation Loss over Epochs.

On the testing set, DenseNet correctly classified **23** Parkinson's cases and **28** healthy cases. However, it misclassified **7** Parkinson's cases and **2** healthy cases, suggesting room for improvement in distinguishing between the two classes.

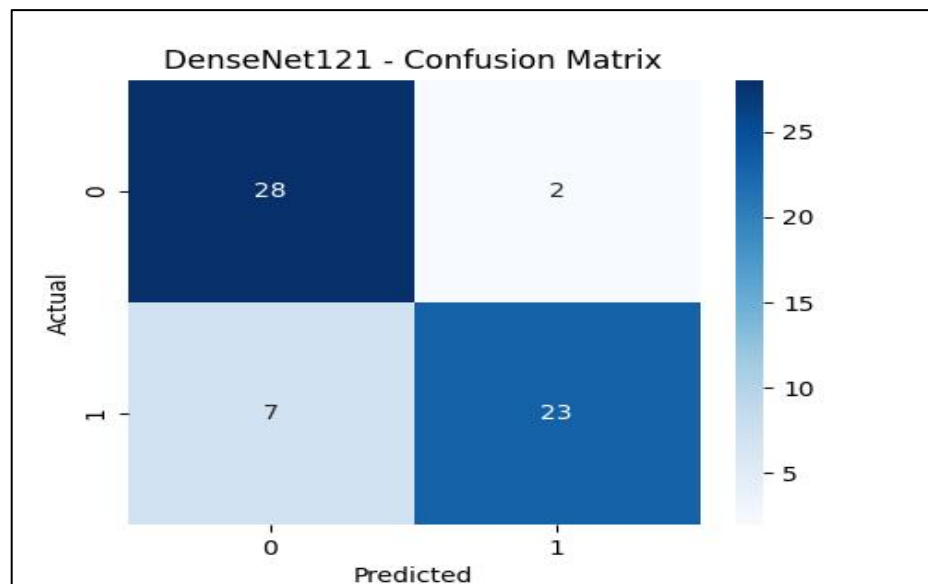
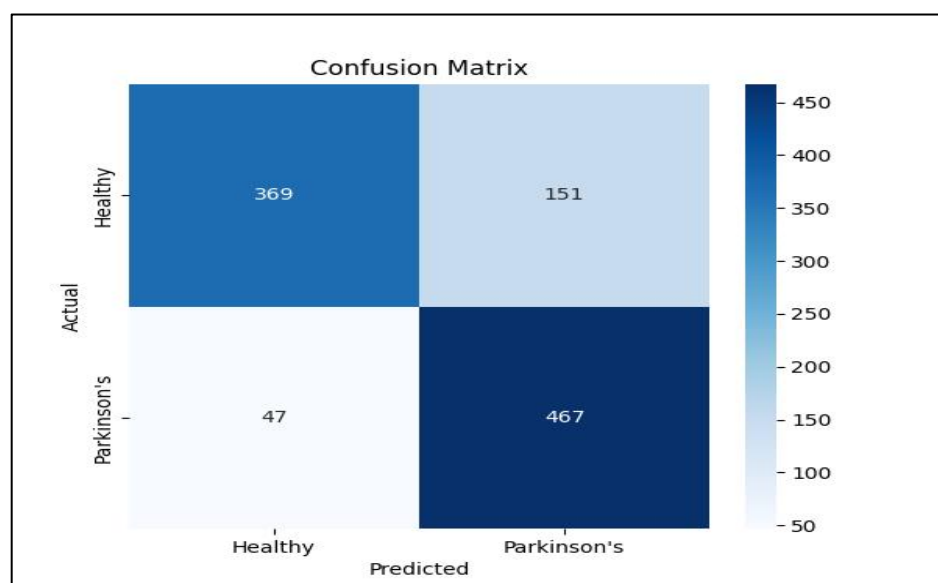


Fig.5.18 Confusion Matrix of DenseNet121 Model on Testing Dataset.

On the Validation set, DenseNet correctly classified **467** Parkinson's cases and **369** healthy cases. However, it misclassified **151** Parkinson's cases and **47** healthy cases, suggesting room for



improvement in distinguishing between the two classes.

Fig.5.19 Confusion Matrix of DenseNet121 on Validation Dataset

The DenseNet model achieved an AUC of **0.93** on the testing set and **0.92** on the validation set, demonstrating strong and reliable performance in distinguishing between Parkinson's patients and healthy individuals across both datasets.

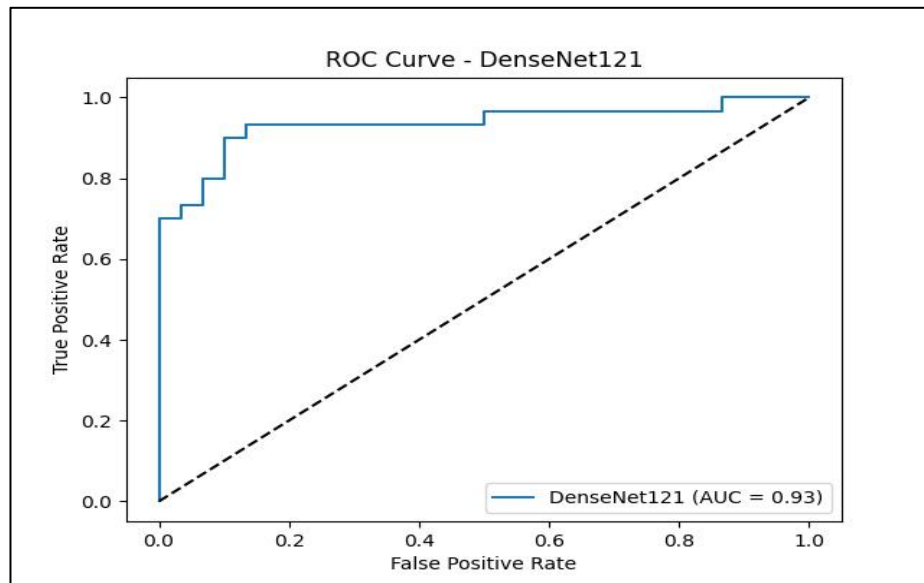


Fig.5.20 ROC Curve of DenseNet121 on Testing Dataset

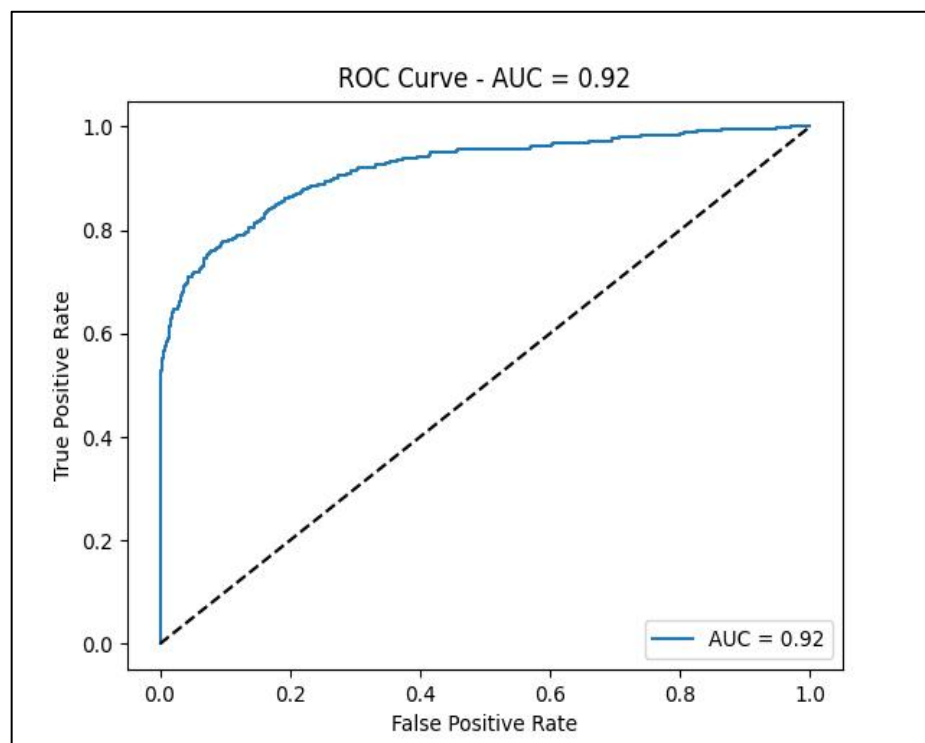


Fig.5.21 ROC Curve of DenseNet121 on Validation Dataset

The accuracy of the DenseNet model steadily improved over epochs, showing effective learning. The training loss consistently decreased, though minor fluctuations in validation loss suggest slight overfitting that could be minimized with further tuning.

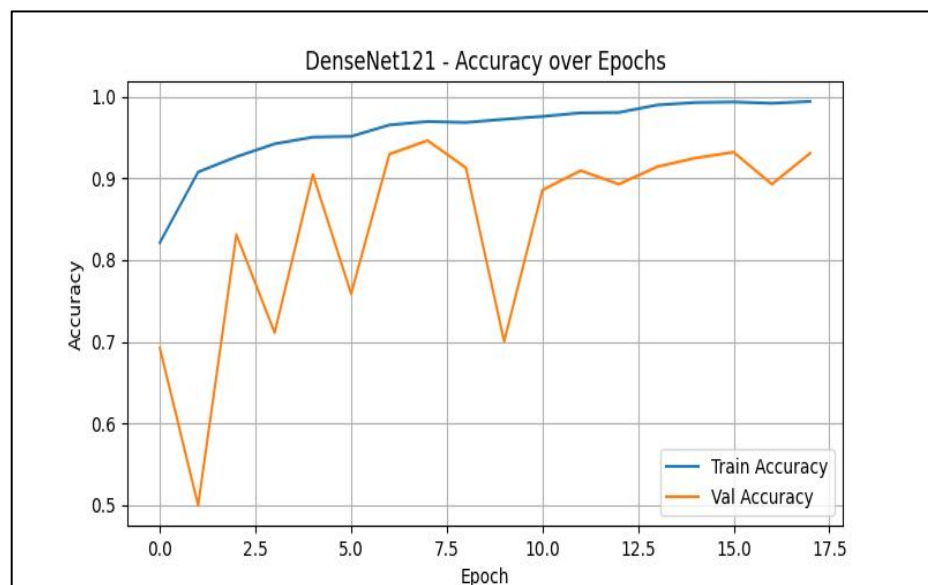


Fig.5.22 DenseNet121 Training vs Validation Accuracy over Epochs.

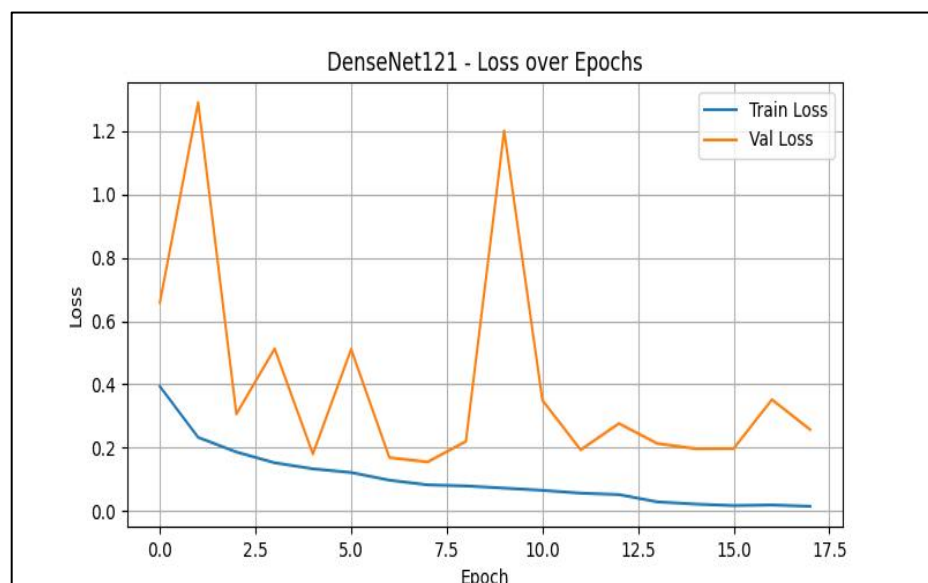


Fig.5.23 DenseNet121 Training vs Validation Loss over Epochs

On the testing set, ResNet correctly classified **23** healthy and **23** Parkinson's cases. However, **7** healthy cases were misclassified as Parkinson's and **7** Parkinson's cases were misclassified as healthy, indicating a balanced but moderate classification performance.

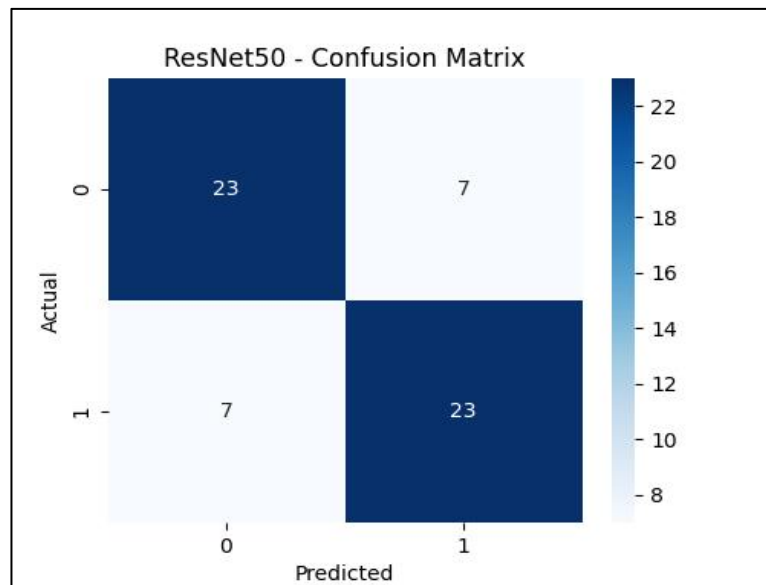


Fig.5.24 Confusion Matrix of ResNet50 on Testing Dataset

On the validation set, ResNet correctly classified **389** healthy cases and **189** Parkinson's cases. However, **131** healthy cases were misclassified as Parkinson's and **155** Parkinson's cases were predicted as healthy, suggesting notable challenges in distinguishing between classes.

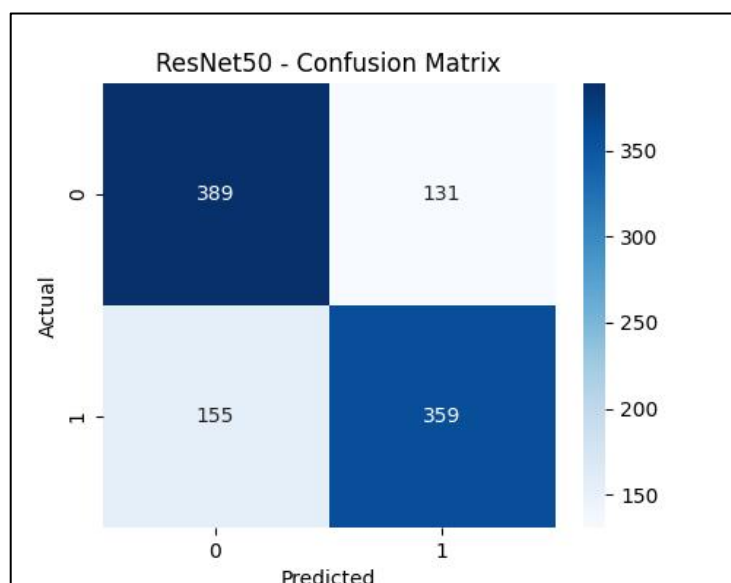


Fig.5.25 Confusion Matrix of ResNet50 on Validation Dataset

The ResNet model achieved an AUC of **0.87** on the testing set and **0.79** on the validation set, indicating good discriminative ability during testing but a noticeable drop in generalization on unseen data.

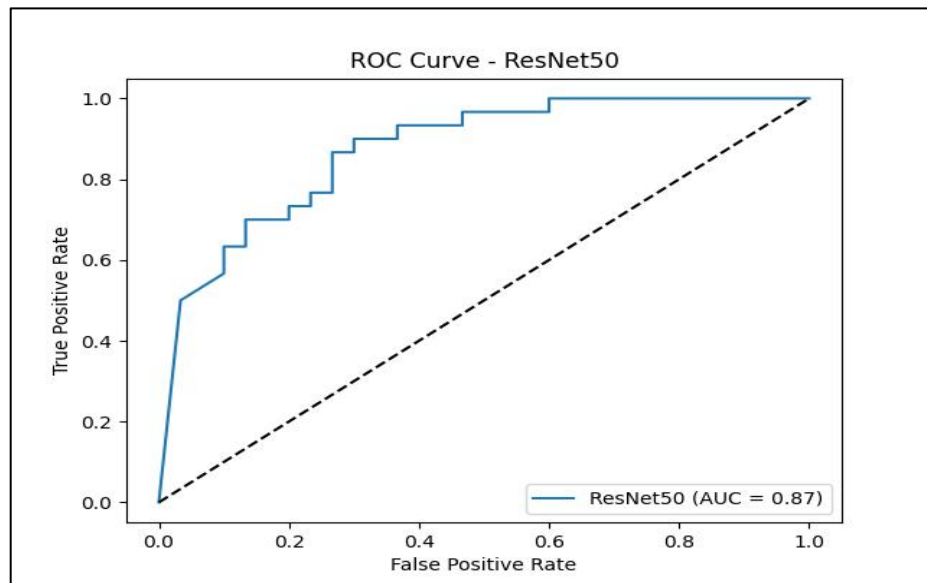


Fig.5.26 ROC Curve of ResNet50 on Testing Dataset

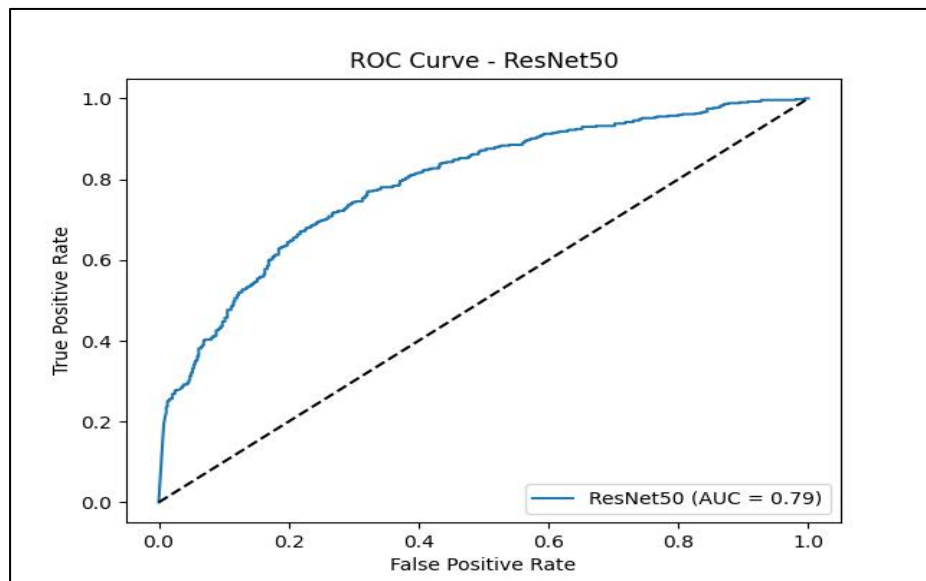


Fig.5.27 ROC Curve of ResNet50 on Validation Dataset

The accuracy of the ResNet model gradually improved over epochs, reflecting effective learning during training. The training loss consistently decreased, although fluctuations in validation loss suggest the model faced challenges with overfitting on unseen data.

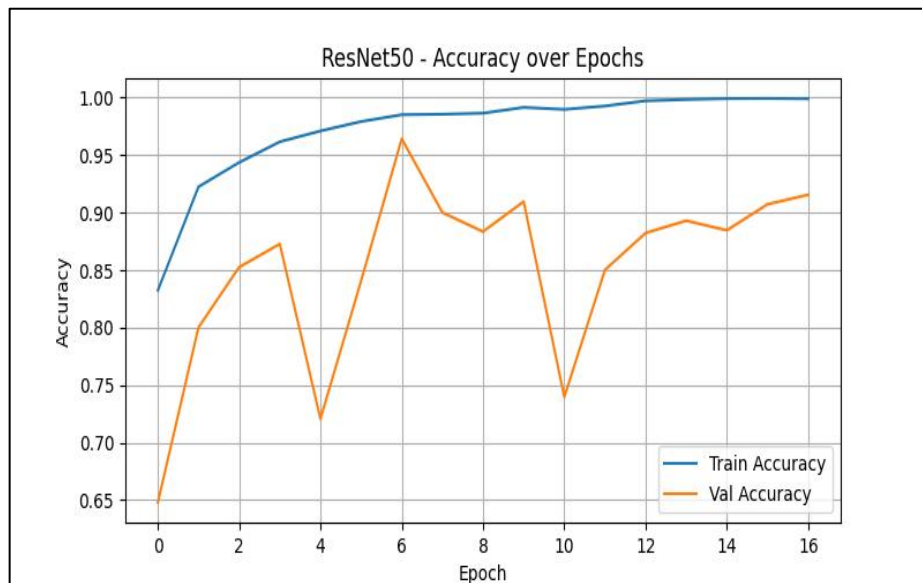


Fig.5.28 ResNet50 Training vs Validation Accuracy over Epochs

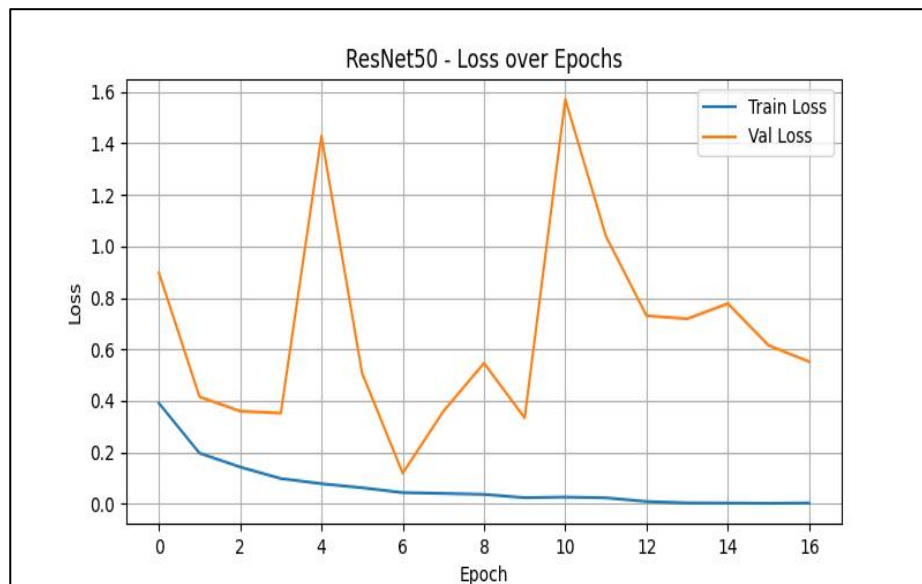


Fig.5.29 ResNet50 Training vs Validation Loss over Epochs

The testing comparison bar graph shows that VGG16 achieved the highest accuracy among all models, followed by DenseNet, ResNet, and CNN. VGG16's superior performance highlights its stronger feature extraction capabilities for image-based Parkinson's detection.

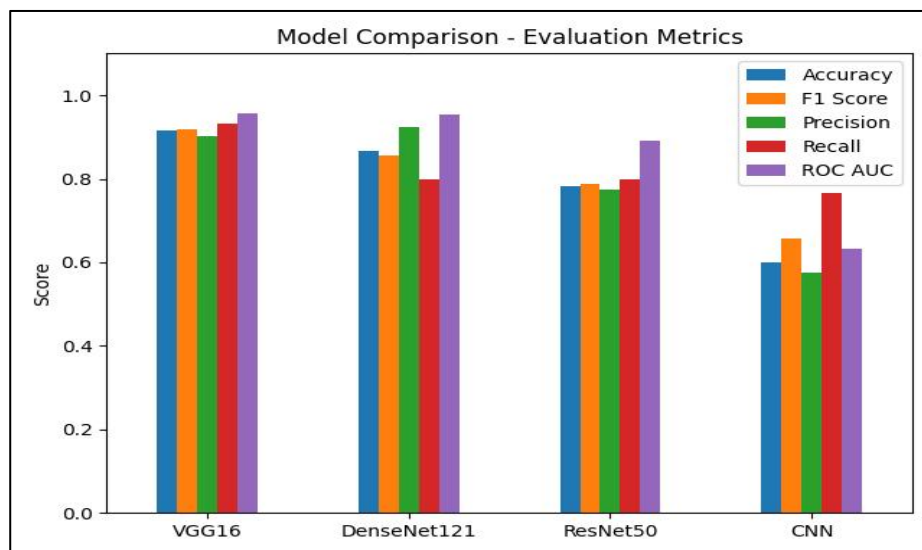


Fig.5.30 Testing Metrics Comparison across Different Models

In the validation comparison bar graph, VGG16 again outperformed other models, maintaining the highest accuracy. DenseNet showed competitive results, while CNN and ResNet had comparatively lower validation performance, indicating slight overfitting or generalization issues.

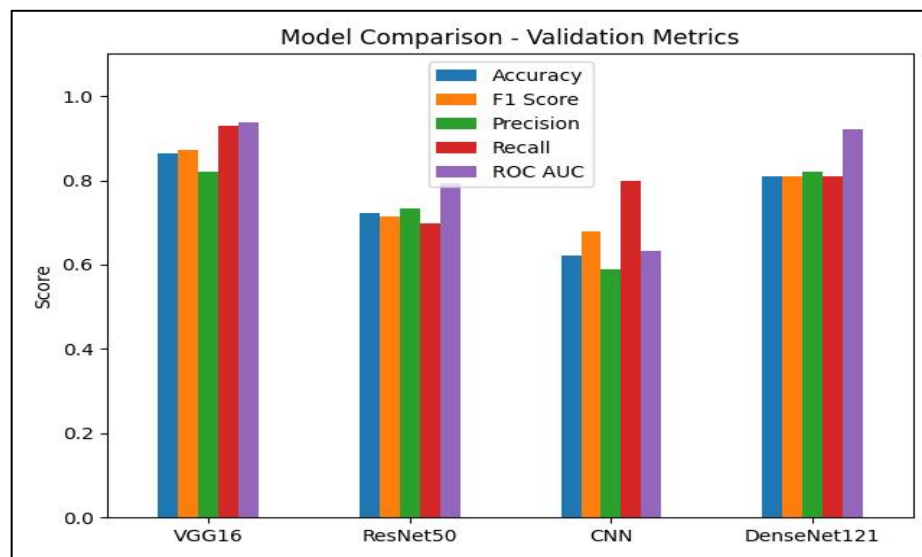


Fig.5.31 Validation Metrics Comparison across Different Models